

UNIT- 1

Design Pattern (Part B)

Design Pattern:

Design patterns are reusable solutions to common problems that arise during software design. They provide a proven development paradigm that helps developers create more efficient, scalable, and maintainable software. Design patterns are established solutions to recurring design problems in software development. They are conceptual frameworks that help developers create more organized, efficient, and maintainable code. Rather than providing exact code implementations, design patterns offer templates or blueprints that can be adapted to different situations.

Benefits of using Design Pattern in software development:

1. **Maintainability:** Design patterns make code more organized, readable, and easier to understand.
2. **Reusability:** Design patterns promote reusability by providing proven solutions to common problems.
3. **Flexibility:** Design patterns make it easier to modify or extend code without affecting other parts of the system.
4. **Scalability:** Design patterns help build scalable systems by allowing for easier addition of new features or components.
5. **Collaboration:** Design patterns provide a common language and framework for developers to communicate and work together more effectively.
6. **Problem-solving:** Design patterns offer proven solutions to common problems, reducing the time and effort spent on solving them.
7. **Modularity:** Design patterns promote modularity by breaking down complex systems into smaller, independent components.
8. **Easier testing:** Design patterns make it easier to write unit tests and integrate tests by providing a clear structure and organization.
9. **Improved communication:** Design patterns provide a common vocabulary and framework for developers to communicate effectively.
10. **Abstraction:** Patterns encapsulate complex processes and expose a simplified interface, making systems easier to use and understand.

Types of Design Patterns:

1. **Creational Patterns:** Creational patterns deal with object creation mechanisms. They aim to provide a way to create objects while hiding the creation logic, rather than instantiating objects directly using constructors. Types of creational patterns are as follows-
 - Singleton
 - Factory Method
 - Abstract Factory
 - Builder
 - Prototype
2. **Structural Patterns:** Structural patterns focus on how classes and objects are composed to form larger structures while keeping the structures flexible and efficient. They help ensure that if one part of a system changes, the entire structure does not need to be altered. Types of structural patterns are as follows-

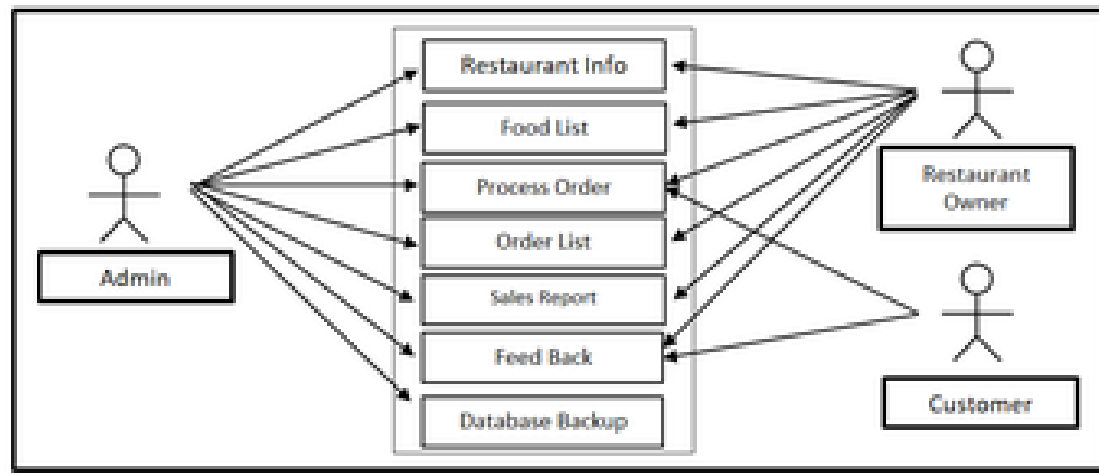
- Adapter
- Bridge
- Composite
- Decorator
- Facade
- Flyweight
- Proxy

3. **Behavioural Patterns:** Behavioural patterns focus on communication between objects, ensuring that they can interact in a flexible, efficient, and scalable way. These patterns help define how objects interact and how to assign responsibilities among objects. Types of Behavioural patterns are as follows-

- Interpreter
- Iterator
- Mediator
- Memento
- Observer
- State
- Strategy
- Template Method
- Visitor

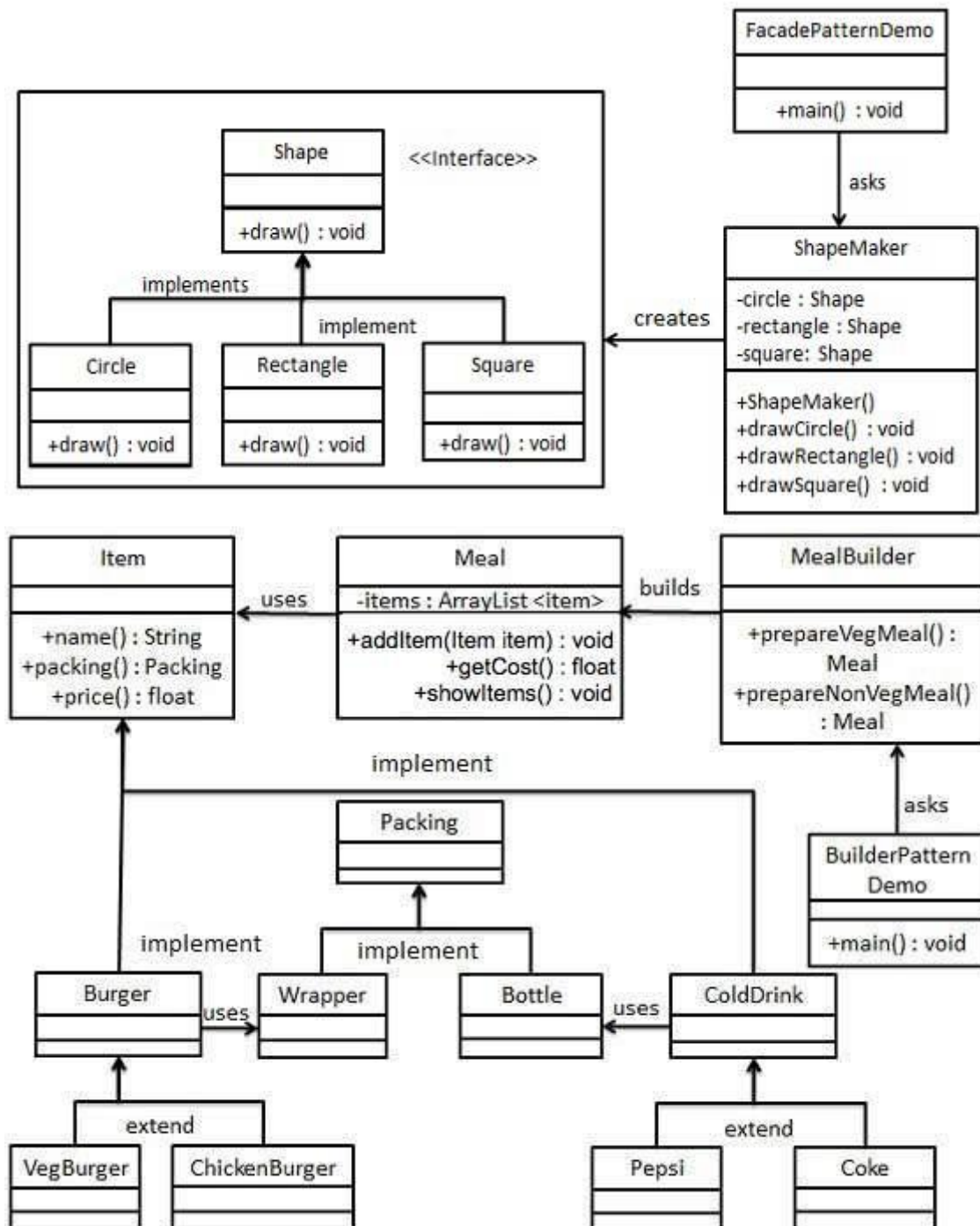
UML Diagrams for design patterns:

Use-case Diagram:



Class Diagram:

Sample class diagram



SOLID PRINCIPLES

SOLID principles are a set of guidelines that help software developers write maintainable, scalable, and flexible code. The acronym SOLID stands for five fundamental principles of object-oriented programming: Single Responsibility Principle, Open/Closed Principle, Liskov Substitution Principle, Interface Segregation Principle, and Dependency Inversion Principle.

These principles were introduced by Robert C. Martin (also known as Uncle Bob) in the early 2000s and have since become widely adopted in the software development community.

1. Single Responsibility Principle

Single Responsibility Principle: The Single Responsibility Principle (SRP) is one of the fundamental principles of object-oriented programming and SOLID design. It states that a class should have only one reason to change, meaning it should have a single responsibility or job.

If a class has more than one responsibility, it becomes coupled. A change to one responsibility results to modification of the other responsibility. A software component must have only one responsibility. It must have only one reason to change. If you need to modify the class for different reasons, this means something (an abstraction) is missing and you need to fix this. If a class has lots of responsibilities this class is often called a "God class" because they know too much or do too much, more than they should do. This type of class is very hard to maintain and the smaller the class, the better.

According to the SRP, a class should be designed to handle a specific task or responsibility. If a class has more than one responsibility, it becomes harder to understand, modify, and maintain over time. When a class needs to be changed, the change should be specific to only one of its responsibilities, not affecting the other responsibilities it handles.

```
# Class that handles the animal's details
class Animal:
    def __init__(self, name, species, sound):
        self.name = name
        self.species = species
        self.sound = sound

    def get_name(self):
        return self.name

    def get_species(self):
        return self.species

    def get_sound(self):
        return self.sound

# Class that handles the speaking functionality (separate class)
class AnimalSpeaker:
    def speak(self, animal: Animal):
        print(f"{animal.get_name()} says {animal.get_sound()}")
```

Benefits of Following SRP

Adhering to the Single Responsibility Principle offers several benefits:

- **Improved Code Organization:** By separating concerns into different classes, the codebase becomes more organized and easier to navigate.
- **Better Maintainability:** When a class has a single responsibility, it is easier to understand its purpose and make changes without unintended side effects.
- **Increased Reusability:** Classes with a single responsibility are more likely to be reusable in different parts of the application or even in other projects.
- **Easier Testing:** Classes with a single responsibility are typically smaller and more focused, making them easier to test in isolation.

2. Open-Closed Principle :

In Open-Closed Principle, Software entities (Classes, modules, functions) should be open for extension, not modification. The Open/Closed Principle means that developers should be able to add new functionality to a class without changing its existing implementation. This can be achieved through techniques such as abstraction,

inheritance, and polymorphism. By adhering to the OCP, developers can create software systems that are more maintainable, scalable, and flexible.

Benefits of Following OCP

Adhering to the Open/Closed Principle offers several benefits:

- **Reduced Risk of Bugs:** By not modifying existing code, the risk of introducing new bugs or breaking existing functionality is minimized.
- **Improved Maintainability:** Code that follows the OCP is easier to maintain and extend, as new features can be added without altering the existing codebase.
- **Enhanced Flexibility:** The use of abstractions and polymorphism allows for more flexible and adaptable designs, making it easier to accommodate changing requirements.

Best Practices for Implementing OCP

To effectively implement the Open/Closed Principle in your codebase, consider the following best practices:

- **Use Abstraction:** Define abstract classes or interfaces to represent common behaviors and allow for different implementations.
- **Leverage Polymorphism:** Use polymorphism to enable different implementations to be used interchangeably.
- **Avoid Tight Coupling:** Design your classes to be loosely coupled, making it easier to extend functionality without modifying existing code.
- **Follow Design Patterns:** Utilize design patterns such as Strategy, Template Method, and Visitor to implement the Open/Closed Principle effectively.

```
#Open-Close Principle
from abc import ABC, abstractmethod

class Animal(ABC):
    @abstractmethod
    def make_sound(self):
        pass

class Dog(Animal):
    def make_sound(self):
        return "Woof!"

class Cat(Animal):
    def make_sound(self):
        return "Meow!"

class AnimalSound:
    def __init__(self, animal: Animal):
        self.animal = animal

    def get_sound(self):
        return self.animal.make_sound()

# Extension without modification
class Cow(Animal):
    def make_sound(self):
        return "Moo!"

dog = Dog()
```

```

cat = Cat()
cow = Cow()

dog_sound = AnimalSound(dog)
cat_sound = AnimalSound(cat)
cow_sound = AnimalSound(cow)

print(dog_sound.get_sound()) # Output: "Woof!"
print(cat_sound.get_sound()) # Output: "Meow!"
print(cow_sound.get_sound()) # Output: "Moo!"

```

3. Liskov Substitution Principle (LSP):

The Liskov Substitution Principle states that objects of a superclass should be replaceable with objects of a subclass without affecting the correctness of the program. In other words, if S is a subtype of T, then objects of type T may be replaced with objects of type S (i.e., objects of type S may be substituted for objects of type T) without altering any of the desirable properties of the program. This principle is based on the concept of subtyping, which is a way to establish a hierarchical relationship between types. A subtype is a type that inherits from another type (the supertype) and can be used in place of the supertype.

The formal definition is:

if S is a subtype of T, then objects of type T may be replaced by objects of type S, without breaking the program.

Benefits of Following LSP

Adhering to the Liskov Substitution Principle offers several benefits:

- **Improved Code Reusability:** By ensuring that subtypes can be substituted for their base types, code that uses the base type can also work with any of its subtypes, promoting code reuse.
- **Enhanced Maintainability:** Code that follows LSP is easier to maintain because it reduces the risk of introducing bugs when modifying or extending the codebase.
- **Better Testability:** LSP makes it easier to write unit tests for classes and their subtypes, as the tests can be written against the base type and should work for all subtypes.

```

#Liskov Substitution
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self) -> float:
        pass

class Rectangle(Shape):
    def __init__(self, width: float, height: float):
        self.width = width
        self.height = height

    def area(self) -> float:
        return self.width * self.height

class Circle(Shape):
    def __init__(self, radius: float):
        self.radius = radius

```

```

    def area(self) -> float:
        return 3.14159 * self.radius * self.radius

class Triangle(Shape):
    def __init__(self, base: float, height: float):
        self.base = base
        self.height = height

    def area(self) -> float:
        return 0.5 * self.base * self.height

def calculat_area(shape:Shape):
    print("The area is:", shape.area())

rectangle = Rectangle(4, 5)
circle = Circle(3)
triangle = Triangle(6, 4)

calculat_area(rectangle) # Output: The area is: 20.0
calculat_area(circle) # Output: The area is: 28.27431
calculat_area(triangle) # Output: The area is: 12.0

```

#Example 2:

```

class Animal:
    def talk(self):
        pass

class Cat(Animal):
    def talk(self):
        return "Meow!"

class Dog(Animal):
    def talk(self):
        return "Woof!"

def greet(animal: Animal):
    print(animal.talk())

cat = Cat()
dog = Dog()
greet(cat) # Output: Meow!
greet(dog) # Output: Woof!

```

#Example 3:

```

class Bird(ABC):
    @abstractmethod
    def fly(self):
        pass

class Penguin(Bird):
    def fly(self):
        print("Penguins can't fly!")

class Parrot(Bird):
    def fly(self):
        print("Parrots can fly!")

```

```
def let_it_fly(bird: Bird):
    bird.fly()

parrot = Parrot()
penguin = Penguin()
let_it_fly(parrot)    # Output: Parrots can fly!
let_it_fly(penguin)  # Output: Penguins can't fly!
```

4. Interface Segregation Principle (ISP)

The Interface Segregation Principle (ISP) is one of the SOLID design principles that help in creating modular and maintainable software systems. It states that "Clients should not be forced to depend on interfaces they do not use." In simpler terms, ISP suggests we break down large interfaces into smaller, more specific ones. This way, clients (classes or modules) only need to depend on the methods they require. This principle promotes loose coupling and high cohesion, making the code more modular, reusable, and easier to maintain.

Benefits of Using Interface Segregation Principle

- **Modular and Reusable Code:** By breaking down large interfaces into smaller, more specific ones, the code becomes more modular and reusable. Classes or modules can implement only the interfaces they need, reducing unnecessary dependencies and making it easier to reuse code across different parts of the system.
- **Reduced Code Complexity:** When classes or modules depend only on the interfaces they need, the code becomes less complex and easier to understand. This is because developers do not have to deal with unnecessary methods or dependencies. These are not relevant to their specific use case.
- **Improved Maintainability:** With smaller and more focused interfaces, it becomes easier to maintain the code. Changes to one interface are less likely to affect other parts of the system, reducing the risk of introducing bugs or breaking existing functionality.
- **Better Testability:** Smaller and more focused interfaces make it easier to write unit tests for individual components. This is because the tests can focus on specific behaviors without being affected by irrelevant methods or dependencies.
- **Increased Flexibility:** By adhering to the ISP, the system becomes more flexible and easier to extend or modify. New features or requirements can be added by creating new interfaces or modifying existing ones without affecting the entire system.

```
#interface segregation
from abc import ABC, abstractmethod

class Printable(ABC):
    @abstractmethod
    def print_document(self, document):
        pass

class Scanner(ABC):
    @abstractmethod
    def scan_document(self):
        pass

class Printer(Printable):
    def print_document(self, document):
```



```

        print("Printing document:", document)

class Copier(Printable, Scanner):
    def print_document(self, document):
        print("Printing document:", document)

    def scan_document(self):
        print("Scanning document")

# Usage:
printer = Printer()
copier = Copier()

printer.print_document("Report.pdf")
copier.print_document("Invoice.pdf")
copier.scan_document()

#In this example, the `Printable` interface focuses solely on printing
functionality, while the `Scanner` interface concentrates on scanning. By
separating the interfaces, we allow clients to depend only on the
functionality they require.

```

#Example 2

```

#interface Segregation example
from abc import ABC, abstractmethod
# Define interfaces
class Swim(ABC):
    @abstractmethod
    def swim(self):
        pass
class Fly(ABC):
    @abstractmethod
    def fly(self):
        pass
# Classes implementing interfaces
class Dolphin(Swim):
    def swim(self):
        print("The dolphin is swimming.")
class Bird(Fly):
    def fly(self):
        print("The bird is flying.")
class Bat(Fly, Swim):
    def fly(self):
        print("The bat is flying.")

    def swim(self):
        print("The bat is swimming.")
def main():
    dolphin = Dolphin()
    bird = Bird()
    bat = Bat()
    dolphin.swim()
    bird.fly()
    bat.fly()
    bat.swim()
if __name__ == "__main__":
    main()

```

5. Dependency Inversion Principle (DIP)

The Dependency Inversion Principle (DIP) is the final principle in the SOLID design principles. It was introduced by Robert C. Martin, a key figure in software engineering, and is important for creating loosely coupled and maintainable software systems. The Dependency Inversion Principle states that:

High-level modules should not depend on low-level modules. Both should depend on abstractions.

Abstractions should not depend on details. Details should depend on abstractions.

In simpler terms, DIP suggests that the high-level policy-setting modules should not depend on the low-level, implementation-specific modules. Instead, both should depend on interfaces or abstract classes. This inversion of dependency helps in achieving a more flexible and resilient architecture.

Benefits of Following DIP

Adhering to the Dependency Inversion Principle offers several benefits:

- **Loose Coupling:** By depending on abstractions rather than concrete implementations, the code becomes less tightly coupled, making it easier to change one part of the system without affecting others.
- **Improved Maintainability:** Changes in low-level modules do not impact high-level modules, making the system easier to maintain and extend.
- **Enhanced Testability:** High-level modules can be tested using mock implementations of the low-level modules, making testing faster and more reliable.
- **Increased Reusability:** High-level modules can be reused in different contexts without needing to change the low-level modules they depend on.

Best Practices for Implementing DIP

- **Use Interfaces and Abstract Classes:** Define interfaces or abstract classes to represent the dependencies between high-level and low-level modules.
- **Dependency Injection:** Use dependency injection to provide the dependencies to a class, rather than having the class create its own dependencies.
- **Avoid Tight Coupling:** Design your classes to be loosely coupled, making it easier to extend functionality without modifying existing code.
- **Follow Other SOLID Principles:** Adhering to the Open/Closed Principle and the Liskov Substitution Principle can help in implementing the Dependency Inversion Principle effectively.

```
#dependency inversion
from abc import ABC, abstractmethod

class DataAccess(ABC):
    @abstractmethod
    def get_data(self):
        pass

class DatabaseAccess(DataAccess):
    def get_data(self):
        # Access data from a database
        pass

class APIDataAccess(DataAccess):
    def get_data(self):
```

```

        # Access data from an API
        pass

class DataManager:
    def __init__(self, data_access):
        self.data_access = data_access

    def fetch_data(self):
        data = self.data_access.get_data()
        # Process the data

# Usage:
database_access = DatabaseAccess()
api_data_access = APIDataAccess()

data_manager_1 = DataManager(database_access)
data_manager_1.fetch_data()

data_manager_2 = DataManager(api_data_access)
data_manager_2.fetch_data()

#dependency inversion example 2

from abc import ABC, abstractmethod

# Abstract base class representing the high-level module
class Animal(ABC):
    @abstractmethod
    def make_sound(self):
        pass

# Low-level modules
class Dog(Animal):
    def make_sound(self):
        return "Woof!"

class Cat(Animal):
    def make_sound(self):
        return "Meow!"

# High-level module using abstractions
class AnimalSound:
    def __init__(self, animal: Animal):
        self.animal = animal

    def get_sound(self):
        return self.animal.make_sound()

# Client code
def main():
    dog = Dog()
    cat = Cat()

    dog_sound = AnimalSound(dog)
    cat_sound = AnimalSound(cat)

    print("Dog sound:", dog_sound.get_sound())
    print("Cat sound:", cat_sound.get_sound())

if __name__ == "__main__":

```

```
main()
```

```
output-
```

```
Dog sound: Woof!
```

```
Cat sound: Meow!
```